

РК2, вариант 24, вариант запроса В

Код rk1.py

```
#24 В

from operator import itemgetter

class Book:

    def __init__(self, id, title):

        self.id = id

        self.title = title


class Chapter:

    def __init__(self, id, name, pages, book_id):

        self.id = id

        self.name = name

        self.pages = pages

        self.book_id = book_id


class BookChapter:

    def __init__(self, book_id, chapter_id):

        self.book_id = book_id

        self.chapter_id = chapter_id


books = [

    Book(1, 'Роман Гарри Поттер'),

    Book(2, 'Роман Мастер и Маргарита'),

    Book(3, 'Учебник по Python'),

    Book(11, 'Учебник по C++'),

    Book(22, 'Роман Гордость и предубеждение'),

]


chapters = [

    Chapter(1, 'Мальчик, который выжил', 50, 1),

    Chapter(2, 'Исчезнувшее стекло', 45, 1),

    Chapter(3, 'Никогда не разговаривайте с неизвестными', 60, 2),
```

```
Chapter(4, 'Введение в язык', 20, 3),  
Chapter(5, 'Введение в данные', 35, 3),  
]
```

```
books_chapters = [  
    BookChapter(1, 1),  
    BookChapter(1, 2),  
    BookChapter(2, 3),  
    BookChapter(3, 4),  
    BookChapter(3, 5),  
    BookChapter(11, 4)  
]
```

```
def get_one_to_many(books, chapters):  
    return [(c.name, c.pages, b.title)  
            for b in books  
            for c in chapters  
            if c.book_id == b.id]
```

```
def get_many_to_many(books, chapters, books_chapters):  
    many_to_many_temp = [(b.title, bc.book_id, bc.chapter_id)  
                          for b in books  
                          for bc in books_chapters  
                          if b.id == bc.book_id]  
    return [(c.name, c.pages, book_title)  
            for book_title, book_id, chapter_id in many_to_many_temp  
            for c in chapters if c.id == chapter_id]
```

```
def task_b1(one_to_many):  
    return [(c_name, b_title)  
            for c_name, _, b_title in one_to_many  
            if c_name.startswith('B')]
```

```
def task_b2(books, one_to_many):
```

```

res_unsorted = []
for b in books:
    b_chapters = [item for item in one_to_many if item[2] == b.title]
    if len(b_chapters) > 0:
        b_pages_min = min([pages for _, pages, _ in b_chapters])
        res_unsorted.append((b.title, b_pages_min))
return sorted(res_unsorted, key=itemgetter(1))

def task_b3(many_to_many):
    return sorted(many_to_many, key=itemgetter(0))

```

Код test.py

```

import unittest

from rk1 import books, chapters, books_chapters, get_one_to_many,
get_many_to_many, task_b1, task_b2, task_b3

class TestBookAnalytics(unittest.TestCase):

    def setUp(self):
        self.one_to_many = get_one_to_many(books, chapters)
        self.many_to_many = get_many_to_many(books, chapters,
books_chapters)

    def test_task_b1(self):
        result = task_b1(self.one_to_many)
        expected = [
            ('Введение в язык', 'Учебник по Python'),
            ('Введение в данные', 'Учебник по Python')
        ]
        self.assertEqual(result, expected)

    def test_task_b2(self):
        result = task_b2(books, self.one_to_many)
        expected = [
            ('Учебник по Python', 20),

```

```

        ('Роман Гарри Поттер', 45),
        ('Роман Мастер и Маргарита', 60)
    ]

    self.assertEqual(result, expected)

def test_task_b3(self):
    result = task_b3(self.many_to_many)
    expected = [
        ('Введение в данные', 35, 'Учебник по Python'),
        ('Введение в язык', 20, 'Учебник по Python'),
        ('Введение в язык', 20, 'Учебник по C++'),
        ('Исчезнувшее стекло', 45, 'Роман Гарри Поттер'),
        ('Мальчик, который выжил', 50, 'Роман Гарри Поттер'),
        ('Никогда не разговаривайте с неизвестными', 60, 'Роман
Мастер и Маргарита')
    ]

    self.assertEqual(result, expected)

if __name__ == '__main__':
    unittest.main()

```

Результат выполнения

```

sasku@kubik:~/IU5_PCPL_Course$ python3 test.py
...
-----
Ran 3 tests in 0.001s

OK

```